Universidade Estadual de Santa Cruz (UESC)

Etapa 2 — Benchmark de Geração do Conjunto de Mandelbrot (Escape-Time): Implementação paralela em memória distribuída com MPI.

Relatório acadêmico — DEC107 — Processamento Paralelo

Autores: Cibelle Sousa Rodrigues, Lucas Braga Orihuela

Disciplina: DEC107 — Processamento Paralelo

Data: 30/10/2026

1. Introdução

O objetivo desta segunda etapa do projeto é avaliar o aprendizado prático dos conceitos de processamento paralelo em memória distribuída utilizando a biblioteca MPI. Enquanto a Etapa 1 explorou o paralelismo em memória compartilhada com OpenMP, em que todas as threads enxergam a mesma matriz de resultados, esta etapa parte de um cenário diferente, no qual cada processo possui o seu próprio espaço de endereçamento e nada é compartilhado implicitamente. Toda informação que precisa circular entre os processos deve ser enviada de forma explícita por troca de mensagens.

O Conjunto de Mandelbrot é definido como o conjunto de números complexos c para os quais a sequência iterativa z_{n+1} = z_n² + c, com z_0 = 0, permanece limitada. Na implementação computacional, o algoritmo de escape-time verifica cada ponto do plano complexo mapeado, iterando essa fórmula até que o valor de |z| ultrapasse um raio de escape pré-definido, que neste projeto é |z| > 2, verificado na forma equivalente |z|² > 4. O número de iterações necessárias para que o ponto escape desse raio, limitado a um teto máximo MAX_ITER, dita a sua coloração na geração da imagem fractal.

O custo computacional de cada pixel é diretamente proporcional ao seu escape-time. Pixels localizados fora do conjunto escapam em poucas iterações, ao passo que pixels situados na fronteira ou no interior do conjunto consomem o limite máximo de iterações. Essa heterogeneidade geométrica da carga é o que torna o problema interessante como benchmark, pois ele é embaraçosamente paralelo, sem nenhuma dependência de dados entre pixels, e ao mesmo tempo fortemente desbalanceado.

Em memória distribuída esse desbalanceamento ganha uma consequência adicional que não existia na Etapa 1. Em OpenMP, uma política de escalonamento dinâmica redistribui iterações entre threads a um custo de sincronização baixo, porque a fila de trabalho e a matriz de saída residem na mesma memória. Em MPI, redistribuir trabalho significa trocar mensagens, e devolver o resultado ao processo responsável pela escrita significa transferir blocos de linhas pela rede ou pela memória do sistema. A escolha da estratégia de distribuição passa, portanto, a envolver um compromisso entre dois custos que competem entre si, o custo do desbalanceamento e o custo da comunicação.

Esse compromisso justifica a estrutura do experimento desta etapa. Foram implementadas três estratégias de distribuição do domínio da imagem, com granularidades distintas, e todas foram medidas sob os mesmos parâmetros, no mesmo binário e no mesmo ambiente. A decomposição estática por faixas contíguas representa o custo de comunicação mínimo com o pior balanceamento possível. A distribuição cíclica de blocos de linhas mantém o custo de comunicação baixo e melhora o balanceamento sem nenhuma negociação em tempo de execução. O esquema master-worker adota balanceamento dinâmico, ao preço de dedicar um processo à coordenação e de gerar tráfego de mensagens proporcional ao número de tarefas.

Por fim, a análise rigorosa de desempenho em memória distribuída exige atenção a um ponto que a Etapa 1 não precisou enfrentar com o mesmo peso. Conforme registrado em aula, o tempo total de execução de um programa paralelo se decompõe em tempo de computação, tempo ocioso e tempo de comunicação. Separar essas três parcelas é o que permite afirmar se uma estratégia perdeu desempenho por distribuir mal o trabalho, por deixar processos esperando, ou por pagar caro na troca de mensagens. Por isso, além das métricas obrigatórias de speedup, eficiência e fator de balanceamento de carga, este relatório reporta também a granularidade e o custo de comunicação medidos diretamente na execução.

2. Metodologia

2.1 Ambiente de Execução

Todos os experimentos foram conduzidos em um notebook com sistema operacional Arch Linux, kernel Linux 7.2.9-arch1-1 x86_64, arquitetura x86_64. O compilador utilizado foi gcc (GCC) 16.2.1 20260810, e a implementação de MPI foi mpirun (Open MPI) 5.0.11. A versão paralela foi compilada com mpicc, que invoca o compilador do sistema adicionando as bibliotecas de MPI, e a versão sequencial de referência foi compilada diretamente com gcc. Ambas usaram as flags -O3 -std=c11, mantendo exatamente as mesmas condições de otimização adotadas na Etapa 1.

O processador é um Intel(R) Core(TM) 7 240H com arquitetura híbrida, um aspecto determinante para a interpretação dos resultados:

- **10 núcleos físicos e 16 CPUs lógicas**, configurando uma combinação de núcleos de performance, os P-cores, que possuem Hyper-Threading, e núcleos de eficiência, os E-cores, que não possuem Hyper-Threading.
- Caches L1d de 416 KiB, L1i de 448 KiB, L2 de 9,5 MiB e L3 de 24 MiB em instância única, com um único nó NUMA.

Essa heterogeneidade é relevante nesta etapa por um motivo específico. Como todos os processos MPI são executados na mesma máquina física, um processo alocado em um E-core executa a mesma quantidade de trabalho em mais tempo do que um processo alocado em um P-core. O fator de balanceamento de carga passa, então, a medir dois efeitos sobrepostos, o desbalanceamento geométrico do fractal e a assimetria do hardware.

Para controle da frequência, o governador foi fixado em performance durante toda a coleta final de dados, por meio do wrapper ***roda_com_performance_mpi.sh***, que liga o modo performance antes dos experimentos e o restaura automaticamente ao final. Essa decisão repete o procedimento adotado na Etapa 1 e é justificada empiricamente. Em uma coleta preliminar conduzida com o governador em powersave, o tempo serial de referência no input padrão ficou em torno de 7,60 s, contra 5,66 s na coleta final em performance, uma diferença de aproximadamente 37%. Medir a Etapa 2 sob um governador diferente do utilizado na Etapa 1 inviabilizaria a comparação entre os dois paradigmas, que é exigida pelo enunciado. Para impedir que essa contaminação volte a ocorrer, o script de experimentos recusa-se a executar quando detecta que o governador não está em performance.

2.2 Implementação Sequencial de Referência

A implementação sequencial base foi desenvolvida na linguagem C e é a mesma utilizada na Etapa 1, sem nenhuma alteração. Ela cumpre dois papéis, serve como linha de base T(n) para o cálculo das métricas de desempenho e serve como referência de corretude para a validação das versões paralelas. O algoritmo processa a matriz da imagem linha por linha utilizando laços aninhados simples, onde a função *escape_time* calcula o número de iterações necessárias para cada ponto do plano complexo.

A saída do algoritmo gera um arquivo binário contendo a matriz de contagens em inteiros de 32 bits, que é comparado byte a byte com o resultado da versão paralela, e um arquivo PGM para inspeção visual. Reutilizar a mesma implementação sequencial das duas etapas garante que qualquer diferença de desempenho observada seja atribuível ao paradigma de paralelização, e não a uma mudança no núcleo de cálculo.

2.3 Implementação Paralela com MPI

A versão paralela em MPI preserva integralmente a função *escape_time* e o mapeamento de pixel para plano complexo da versão sequencial, incluindo a ordem das operações em ponto flutuante. Isso é o que garante igualdade bit a bit entre as saídas, e isola a diferença de desempenho na forma como o trabalho é dividido e como o resultado é coletado.

As funções de MPI utilizadas e a justificativa de cada escolha são as seguintes:

- **MPI_Init** e **MPI_Finalize**, que delimitam a região paralela e inicializam o ambiente de execução.
- **MPI_Comm_rank** e **MPI_Comm_size**, que informam a cada processo o seu identificador e o número total de processos, permitindo que cada um calcule sozinho qual parte do domínio lhe cabe.
- **MPI_Gatherv**, utilizada para a coleta do resultado nas estratégias estáticas. A escolha por Gatherv, e não por Gather, é deliberada. Quando a altura da imagem não é múltipla do número de processos, as faixas têm tamanhos diferentes, e Gather exige blocos de tamanho idêntico. Gatherv aceita um vetor de contagens e um vetor de deslocamentos, o que permite tratar o caso geral sem exigir que a resolução seja divisível pelo número de processos.
- **MPI_IN_PLACE** na raiz da estratégia bloco, que evita uma cópia intermediária. Como o rank 0 já calcula diretamente dentro da matriz final, não faz sentido enviar esses dados para ele mesmo.
- **MPI_Send** e **MPI_Recv**, que implementam o diálogo entre o coordenador e os trabalhadores no esquema master-worker. A recepção no coordenador usa MPI_ANY_SOURCE, pois o coordenador não sabe de antemão qual trabalhador ficará livre primeiro, e é exatamente essa indeterminação que produz o balanceamento dinâmico.
- **MPI_Barrier**, usada para alinhar todos os processos imediatamente antes do início da medição de tempo. Sem essa barreira, processos que iniciam mais cedo contaminariam o cronômetro, e em sala de aula foi visto que a sincronização dos processos no início do código é uma etapa necessária para obter tomadas de tempo confiáveis.
- **MPI_Wtime**, que fornece a medição de tempo de alta resolução recomendada para programas MPI.
- **MPI_Gather**, utilizada ao final para recolher na raiz os tempos individuais de computação e de comunicação de cada processo, além do número de linhas processadas. Esses vetores são a base do diagnóstico de desbalanceamento, cumprindo nesta etapa o mesmo papel que os vetores por thread cumpriram na Etapa 1.

Três decisões de projeto merecem registro explícito. A primeira é que a unidade de trabalho é a linha da imagem, e não o pixel. Cada linha é indivisível e atribuída a um único processo, o que reduz o volume de mensagens e mantém os blocos transferidos contíguos em memória. A segunda é que a configuração não é distribuída por broadcast. Como o mpirun entrega o mesmo argv a todos os processos, cada um realiza o próprio parse dos argumentos, o que dispensa uma rodada de comunicação na inicialização. A terceira é que a estratégia de distribuição é escolhida em tempo de execução pela opção de linha de comando *-e*, exatamente como a Etapa 1 escolhia a política pela variável *OMP_SCHEDULE*. Assim, o mesmo binário serve a todos os cenários, e nenhuma diferença de compilação pode contaminar a comparação entre estratégias.

No esquema master-worker há ainda uma otimização de protocolo que vale descrever. Cada mensagem enviada por um trabalhador ao coordenador carrega um cabeçalho com a linha inicial e a quantidade de linhas, seguido dos dados calculados. O primeiro contato de cada trabalhador vai com contagem zero, sinalizando apenas um pedido de trabalho. Dessa forma, o resultado de uma tarefa e o pedido da tarefa seguinte viajam na mesma mensagem, o que reduz pela metade o número de mensagens em relação a um protocolo que separasse entrega e requisição.

2.4 Estratégias de Distribuição Avaliadas

Foram implementadas três estratégias de distribuição do domínio da imagem, selecionáveis em tempo de execução. Elas foram escolhidas para cobrir o espectro entre comunicação mínima com balanceamento ruim e balanceamento ótimo com comunicação elevada:

Tabela 1 - Estratégias de distribuição implementadas em MPI

ESTRATÉGIA	IDEIA	COLETA DO RESULTADO	EQUIVALENTE NA ETAPA 1
bloco	Cada processo recebe uma faixa contígua de linhas, e as linhas excedentes vão para os primeiros ranks	MPI_Gatherv direto na matriz final, com MPI_IN_PLACE na raiz	static sem chunk
ciclico	Blocos de chunk linhas distribuídos em round-robin entre os processos	MPI_Gatherv em buffer intermediário, seguido de reordenação na raiz	static com chunk
mestre	O rank 0 apenas coordena e entrega chunk linhas a quem estiver livre	Cada trabalhador devolve o resultado junto com o pedido seguinte	dynamic com chunk

A decisão de incluir chunk de valor 1 nas estratégias cíclica e master-worker é deliberada e repete o raciocínio validado na Etapa 1. A região de custo máximo do

fractal é contígua e concentrada, e a granularidade fina é o mecanismo que dilui essa região entre os processos. Comparar chunk 1 contra chunk 16 permite quantificar o compromisso entre diluição da carga e custo de comunicação, que nesta etapa é mais caro do que era em memória compartilhada.

Duas propriedades estruturais do esquema master-worker precisam ser registradas desde já, porque condicionam a leitura de todos os resultados. A primeira é que o rank 0 não calcula nada, de modo que com p processos apenas p − 1 produzem trabalho útil. A segunda, que decorre da primeira, é que a estratégia exige no mínimo dois processos e por isso não aparece nos cenários com p = 1.

2.5 Métricas de Desempenho

Foram adotadas exclusivamente as métricas definidas em aula, caracterização de desempenho em programas paralelos. A notação seguiu o material didático da disciplina, com n representando o tamanho do problema e p o número de processos.

- **Tempo de execução:** T(n, p) é o tempo total de execução, medido do instante em que o primeiro processo começa até que o último processo complete a sua última instrução. Na prática, é o tempo decorrido entre a MPI_Barrier inicial e o instante em que a matriz completa está disponível na raiz, ou seja, computação mais coleta. O tempo de escrita em disco é medido separadamente e reportado à parte, conforme exige o enunciado.
- **Speedup:** S(n, p) = T(n) / T(n, p), onde T(n) é o tempo da versão sequencial de referência para o mesmo tamanho de problema, medido com 7 repetições.
- **Eficiência:** E(n, p) = S(n, p) / p = T(n) / (p · T(n, p)), que quantifica o aproveitamento de cada processo.
- **Fator de balanceamento de carga:** F_LB(n) = (T_max(n) − T_min(n)) / T_max(n), onde T_max é o tempo do último processo a terminar e T_min é o tempo do primeiro. O valor 0 indica balanceamento perfeito e valores próximos de 1 indicam desbalanceamento acentuado.
- **Granularidade:** G(n) = T_cp(n) / T_cm(n), que expressa a relação entre a quantidade de computação e a quantidade de comunicação. Valores altos indicam granularidade grossa e valores baixos indicam granularidade fina.
- **Sobrecarga:** T_0(n, p) = p · T(n, p) − T(n), que mede o trabalho adicional introduzido pela paralelização em relação ao programa sequencial equivalente.

Conforme pede o modelo de relatório, é necessário indicar explicitamente o que foi considerado em cada tempo e como ele foi medido. Os critérios adotados foram os seguintes:

- **T_max, T_min e T_cp:** abrange exclusivamente as chamadas à função escape_time e o laço que percorre os pixels atribuídos ao processo. Não inclui nenhuma chamada de MPI nem a escrita em disco.

- **T_cm:** é o tempo acumulado dentro de chamadas de MPI. É importante registrar que essa medida agrega duas das três parcelas descritas no material da disciplina, o tempo de comunicação propriamente dito e o tempo ocioso de espera, pois um processo que termina cedo e bloqueia em MPI_Gatherv permanece dentro da chamada até que o processo mais lento chegue. Essa distinção é decisiva para a leitura da granularidade e será retomada na *Seção 4*.

- **Escopo do fator de balanceamento:** o fator foi calculado apenas sobre os processos que efetivamente calculam. No esquema master-worker, o rank 0 coordena sem computar, e incluí-lo produziria T_min igual a zero e um fator artificialmente igual a 1 em todos os casos, o que não descreveria o balanceamento real entre os trabalhadores.

- **Custo líquido de comunicação:** definido como T(n, p) − T_max, ou seja, o que resta do tempo total depois de descontar o trabalho útil do processo mais lento. Diferentemente de T_cm, essa medida não contabiliza a espera dos processos rápidos, e por isso é a estimativa mais honesta do custo líquido de comunicação e sincronização.

Cabe uma observação de continuidade metodológica. A Etapa 1 utilizou, para o fator de balanceamento, a razão entre o tempo máximo e o tempo médio entre threads, cujo valor ideal é 1. A definição adotada nesta etapa é a combinada em sala de aula, F_LB(n) = (T_max − T_min) / T_max, cujo valor ideal é 0. As duas grandezas medem o mesmo fenômeno, porém em escalas distintas e não são numericamente comparáveis entre si. Por esse motivo, sempre que este relatório confrontar o balanceamento das duas etapas, a comparação será feita no plano qualitativo, pela ordenação relativa das estratégias, e nunca pela diferença absoluta entre os valores.

2.6 Planejamento dos Experimentos

Os experimentos cobrem os três blocos obrigatórios do enunciado, aplicados a cada uma das cinco configurações de estratégia e chunk avaliadas:

- **Escalabilidade forte:** resolução fixa de 4096 × 4096 e MAX_ITER = 1000, variando o número de processos em 1, 2, 4, 8 e 16.

- **Escalabilidade fraca:** trabalho por processo aproximadamente constante, escalando a resolução junto com o número de processos segundo $N = 4096 \cdot \sqrt{p}$. As resoluções foram 4096^2, 5793^2, 8192^2, 11585^2 e 16384^2 para 1, 2, 4, 8 e 16 processos, respectivamente. Esses são exatamente os mesmos pontos utilizados na

Etapa 1, o que permite comparação direta entre os dois paradigmas, e incluem as três resoluções citadas no enunciado.

- **Caso de desbalanceamento acentuado:** região do vale dos cavalos-marinhos, centrada em -0,743643887 e 0,131825904, com largura real de $3{,}0 \times 10^{-3}$ e MAX_ITER = 5000, para 2, 4, 8 e 16 processos. Essa região concentra grande quantidade de pixels de custo elevado e evidencia os problemas de distribuição de trabalho.

Uma decisão importante distingue o planejamento da escalabilidade fraca nesta etapa. A métrica de eficiência definida em sala de aula foi $E(n, p) = T(n) / (p \cdot T(n, p))$, e o numerador exige o tempo sequencial no mesmo tamanho de problema *n* do caso paralelo. Como na escalabilidade fraca o tamanho do problema muda a cada ponto, não é suficiente dispor de um único baseline sequencial em 4096^2. Por esse motivo, foi medido um baseline sequencial independente para cada uma das cinco resoluções, o que torna possível aplicar a mesma fórmula de eficiência em todos os cenários do relatório, sem recorrer a nenhuma definição externa ao material da disciplina.

Cada configuração paralela dos cenários de escalabilidade forte e de zoom foi executada 5 vezes, e cada configuração de escalabilidade fraca foi executada 3 vezes, reportando-se média e desvio padrão. Os baselines sequenciais do input padrão e do zoom foram medidos com 7 repetições, e os baselines por resolução da escalabilidade fraca com 3 repetições. Ao todo foram registradas 284 execuções paralelas.

2.7 Validação da Corretude

A corretude foi validada comparando bit a bit a matriz de contagens gerada por cada estratégia paralela contra a matriz da versão sequencial de referência, tanto no input padrão quanto no caso de zoom. A comparação é feita com o utilitário cmp sobre os arquivos binários, o que detecta qualquer divergência de um único byte.

Por se tratar de problema determinístico, com contagens inteiras e aritmética idêntica por pixel, a expectativa do enunciado é de igualdade exata, admitindo-se no máximo divergência em 0,01% dos pixels de fronteira. A validação é executada automaticamente como etapa obrigatória do script de experimentos, que interrompe toda a bateria caso qualquer estratégia diverja da referência. Dessa forma, nenhum número de desempenho pode ser coletado a partir de uma implementação incorreta.

Além da validação automática da coleta final, a implementação foi testada com 1, 2, 3, 5 e 7 processos nas três estratégias, com chunks de 1, 3, 16 e 1000, incluindo alturas de imagem que não são divisíveis pelo número de processos. Esse último caso é o que exercita o tratamento das linhas excedentes e justifica o uso de MPI_Gatherv no lugar de MPI_Gather.

2.8 Procedimento de Coleta

O fluxo de coleta foi integralmente automatizado e executado com o governador de frequência fixado em performance, por meio do wrapper *roda_com_performance_mpi.sh*, na seguinte ordem:

1. Compilação do binário MPI com mpicc e do binário sequencial de referência com *gcc*, ambos com *-O3 -std=c11*.
2. Validação de corretude bit a bit das três estratégias, no input padrão e no caso de zoom, com interrupção da bateria em caso de divergência.
3. Medição dos baselines sequenciais T(n) do input padrão e do zoom.
4. Medição dos baselines sequenciais T(n) para cada resolução da escalabilidade fraca.
5. Execução da matriz de experimentos de escalabilidade forte e do caso de zoom, com saída em *resultados_mpi.csv*.
6. Execução da matriz de escalabilidade fraca, com saída no mesmo arquivo.
7. Pós-processamento e geração dos gráficos com *pos_processa_mpi.py*, com saída em *resultados_agregados_mpi.csv*.

O arquivo *resultados_mpi.csv* registra em seu cabeçalho as condições do ambiente, incluindo sistema operacional, compilador, versão do MPI, modelo de processador e número de CPUs lógicas, atendendo à exigência do enunciado de documentar detalhadamente o ambiente de execução. Cada execução do binário MPI emite uma linha em formato CSV com os tempos e o detalhamento por processo, e a bateria completa levou 36 minutos e 41 segundos para ser concluída.

3. Resultados

Esta seção apresenta os resultados experimentais da Etapa 2, organizados na seguinte ordem: validação de corretude, escalabilidade forte, caso de desbalanceamento acentuado, escalabilidade fraca, comparação com a solução OpenMP da Etapa 1 e síntese dos achados. Os números reportados são médias de 5 repetições para os cenários de escalabilidade forte e de zoom e de 3 repetições para a escalabilidade fraca, acompanhadas do desvio padrão, todos coletados sob governador de frequência fixado em performance.

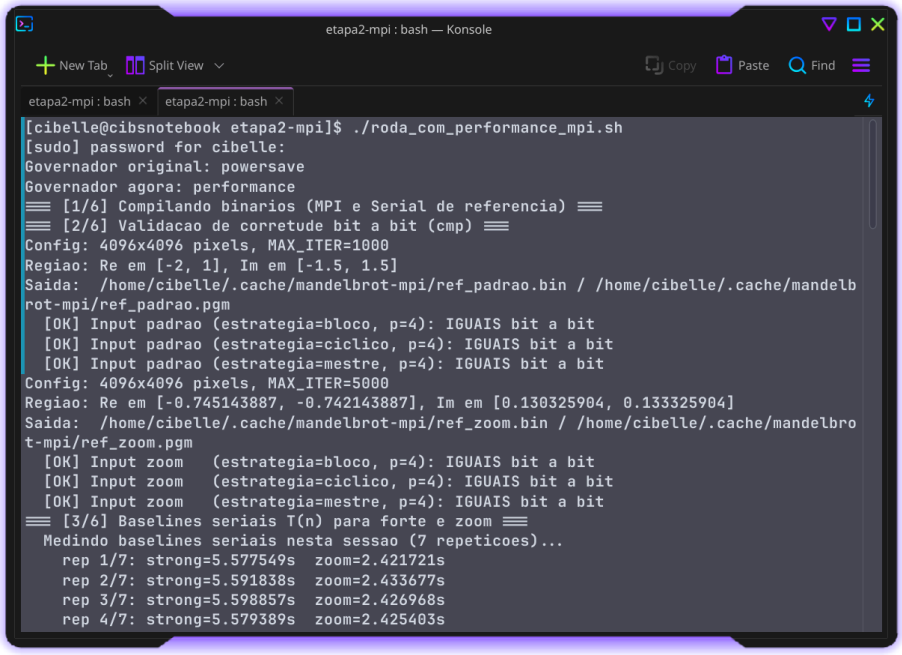
Tabela 2 - Baselines sequenciais de referência T(n)

CENÁRIO	RESOLUÇÃO	MAX_ITER	T(N) MÉDIO (S)	DESVIO PADRÃO (S)	REPETIÇÕES
Input padrão	4096²	1000	5,6583	0,1150	7
Zoom	4096²	5000	2,4502	0,0330	7
Fraca, p = 1	4096²	1000	5,6077	0,0217	3
Fraca, p = 2	5793²	1000	11,1839	0,0343	3
Fraca, p = 4	8192²	1000	22,3916	0,0696	3
Fraca, p = 8	11585²	1000	44,6696	0,1535	3
Fraca, p = 16	16384²	1000	88,7350	0,1828	3

3.1 Validação de Corretude

A corretude foi verificada comparando bit a bit a matriz de contagens produzida por cada uma das três estratégias contra a versão sequencial de referência, tanto no input padrão quanto no caso de zoom. Em todos os seis cenários o resultado foi de igualdade exata, sem nenhum pixel divergente.

Figura 1 - Log da validação de corretude das três estratégias



Saída da etapa de validação do script de experimentos, comparando as três estratégias MPI contra a matriz sequencial de referência no input padrão e no caso de zoom.

Esse desfecho é o esperado para o problema. O valor de cada pixel depende apenas das suas próprias coordenadas no plano complexo, e a função escape_time executa a mesma sequência de operações em ponto flutuante independentemente de qual processo a executa. Como a distribuição de trabalho não altera a ordem das operações dentro de cada pixel, não há reassociação capaz de introduzir divergência numérica. A igualdade exata é, portanto, consequência estrutural do desenho da paralelização, e atende integralmente ao critério da Seção 5.5 do enunciado.

Vale registrar que a validação cobre também o caso em que a altura da imagem não é divisível pelo número de processos, situação em que as faixas atribuídas têm tamanhos diferentes. É esse cenário que justifica o uso de MPI_Gatherv no lugar de MPI_Gather na coleta do resultado.

3.2 Escalabilidade Forte

O experimento de escalabilidade forte mantém a resolução fixa em 4096 × 4096 com MAX_ITER = 1000, variando o número de processos em 1, 2, 4, 8 e 16. A métrica central é o tempo de execução T(n, p), que abrange computação mais coleta do resultado na raiz, e dele derivam o speedup e a eficiência.

3.2.1 Tempo de Execução, Speedup e Eficiência

Tabela 3 - Escalabilidade forte, tempo de execução, speedup e eficiência por estratégia

ESTRATÉGIA	P	T(N,P) (S)	DESVIO (S)	S(N,P)	E(N,P)
bloco	1	5,5734	0,0177	1,02	1,015
bloco	2	2,8129	0,0133	2,01	1,006
bloco	4	2,8829	0,0034	1,96	0,491
bloco	8	2,2642	0,2611	2,50	0,312
bloco	16	1,5765	0,1392	3,59	0,224
ciclico, chunk 1	1	5,6167	0,0116	1,01	1,007
ciclico, chunk 1	2	2,8432	0,0143	1,99	0,995
ciclico, chunk 1	4	1,5200	0,0020	3,72	0,931
ciclico, chunk 1	8	1,0133	0,0344	5,58	0,698
ciclico, chunk 1	16	0,6164	0,0458	9,18	0,574
ciclico, chunk 16	1	5,6019	0,0067	1,01	1,010

ciclico, chunk 16	2	2,8445	0,0084	1,99	0,995
ciclico, chunk 16	4	1,5215	0,0021	3,72	0,930
ciclico, chunk 16	8	1,0186	0,0197	5,56	0,694
ciclico, chunk 16	16	0,6172	0,0239	9,17	0,573
mestre, chunk 1	2	5,6200	0,0114	1,01	0,503
mestre, chunk 1	4	1,9741	0,0023	2,87	0,717
mestre, chunk 1	8	0,9370	0,0192	6,04	0,755
mestre, chunk 1	16	0,4986	0,0034	11,35	0,709
mestre, chunk 16	2	5,6111	0,0103	1,01	0,504
mestre, chunk 16	4	1,9867	0,0245	2,85	0,712
mestre, chunk 16	8	0,9264	0,0171	6,11	0,763
mestre, chunk 16	16	0,5051	0,0282	11,20	0,700

A tabela evidencia três comportamentos nitidamente distintos. A estratégia bloco satura cedo, atingindo speedup de apenas 3,59 com 16 processos, o que corresponde a uma eficiência de 0,224. A estratégia cíclica alcança 9,18 no mesmo ponto, e o esquema master-worker com chunk 1 chega a 11,35, com eficiência de 0,709, o melhor resultado de toda a etapa.

Um ponto merece atenção na leitura da tabela. Com dois processos, o master-worker registra speedup de 1,01 e eficiência de 0,503. Esse valor não indica falha de implementação, mas sim a consequência direta de o rank 0 ser dedicado à coordenação. Com p = 2 existe um único trabalhador, de modo que o tempo paralelo se aproxima do tempo sequencial, e a eficiência se aproxima de 0,5 por definição, já que a fórmula divide o speedup por 2. Esse efeito é discutido em detalhe na *Seção 4*.

Gráfico 1 - Escalabilidade forte, tempo de execução por número de processos

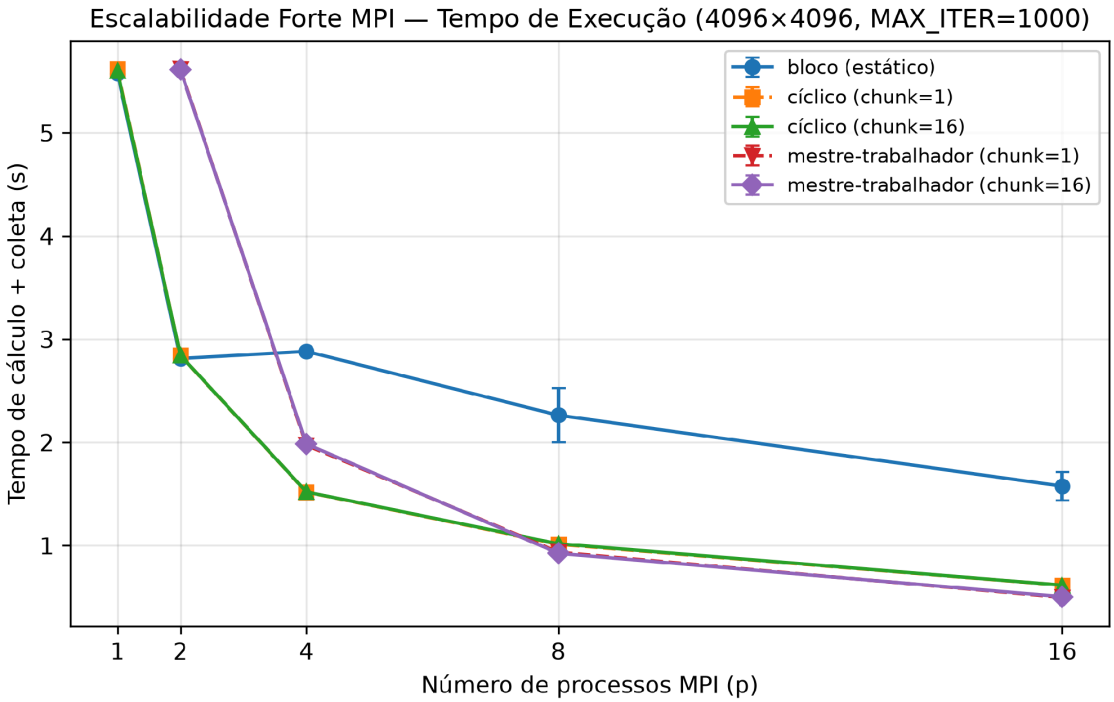
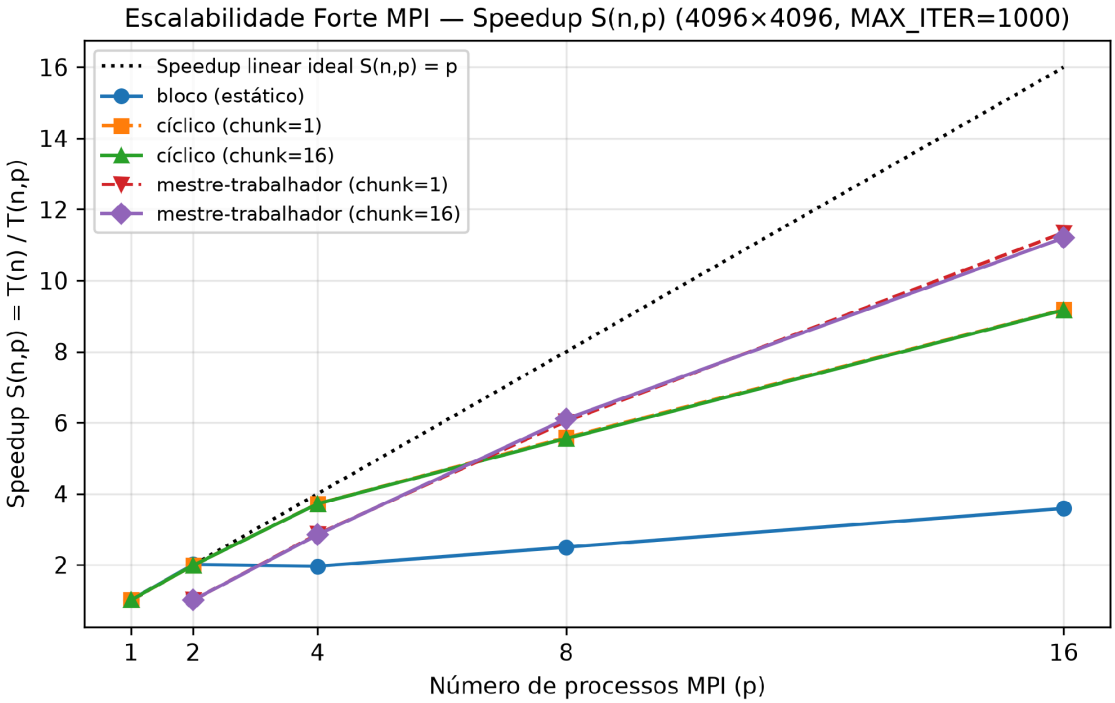
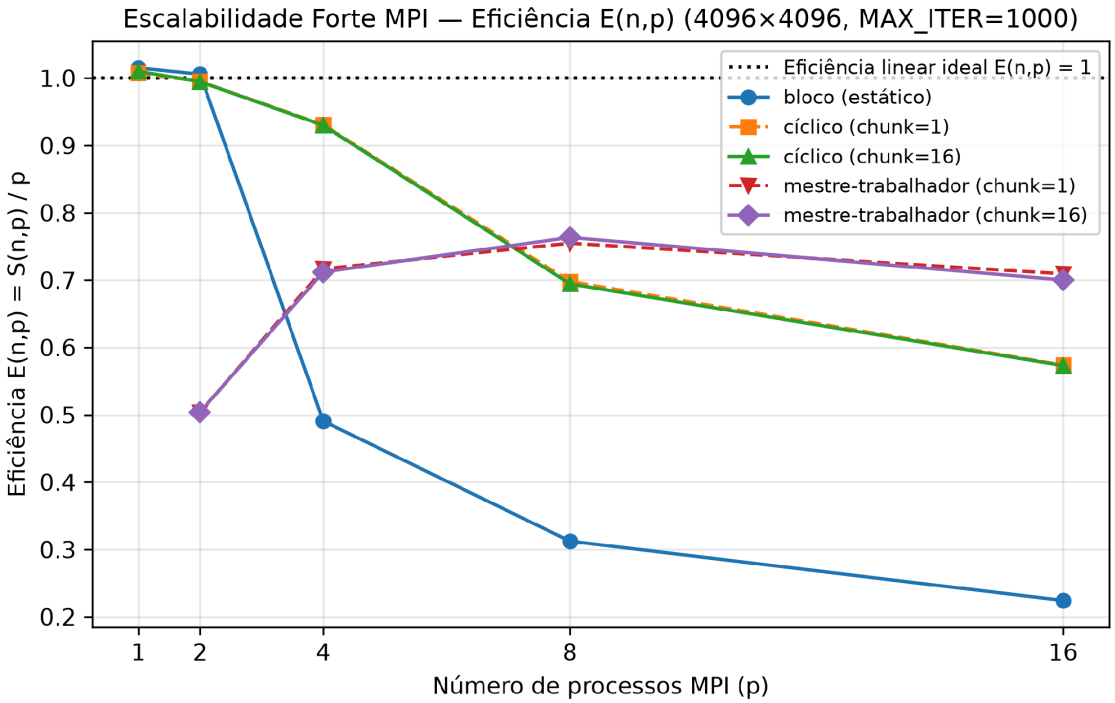

Escalabilidade Forte MPI — Tempo de Execução (4096×4096, MAX_ITER=1000)
bloco (estático)
cíclico (chunk=1)
cíclico (chunk=16)
mestre-trabalhador (chunk=1)
mestre-trabalhador (chunk=16)
Tempo de cálculo + coleta (s)
1
2
3
4
5
1
2
4
8
16
Número de processos MPI (p)

Gráfico 2 - Escalabilidade forte, speedup S(n, p) por número de processos



O Gráfico 2 mostra que nenhuma estratégia acompanha a reta de speedup linear ideal, o que é esperado, e que a separação entre as curvas aumenta com o número de processos. Até dois processos todas as estratégias são praticamente equivalentes. A partir de quatro processos a curva da estratégia bloco se descola das demais e deixa de crescer de forma significativa.

Gráfico 3 - Escalabilidade forte, eficiência E(n, p) por número de processos



No Gráfico 3 cabe registrar uma observação de rigor. Com um único processo, a estratégia bloco apresenta eficiência de 1,015, ligeiramente acima da unidade. Esse valor decorre da variação natural entre execuções, pois o baseline sequencial tem desvio padrão de 0,1150 s, e a diferença observada está dentro de um desvio. Não se trata, portanto, de speedup supralinear real, e sim de ruído de medição.

3.2.2 Fator de Balanceamento de Carga

Para compreender as diferenças de tempo, speedup e eficiência apresentadas acima, é necessário analisar o fator de balanceamento de carga, F_LB(n) = (T_max − T_min) / T_max, em que o valor ideal é 0.

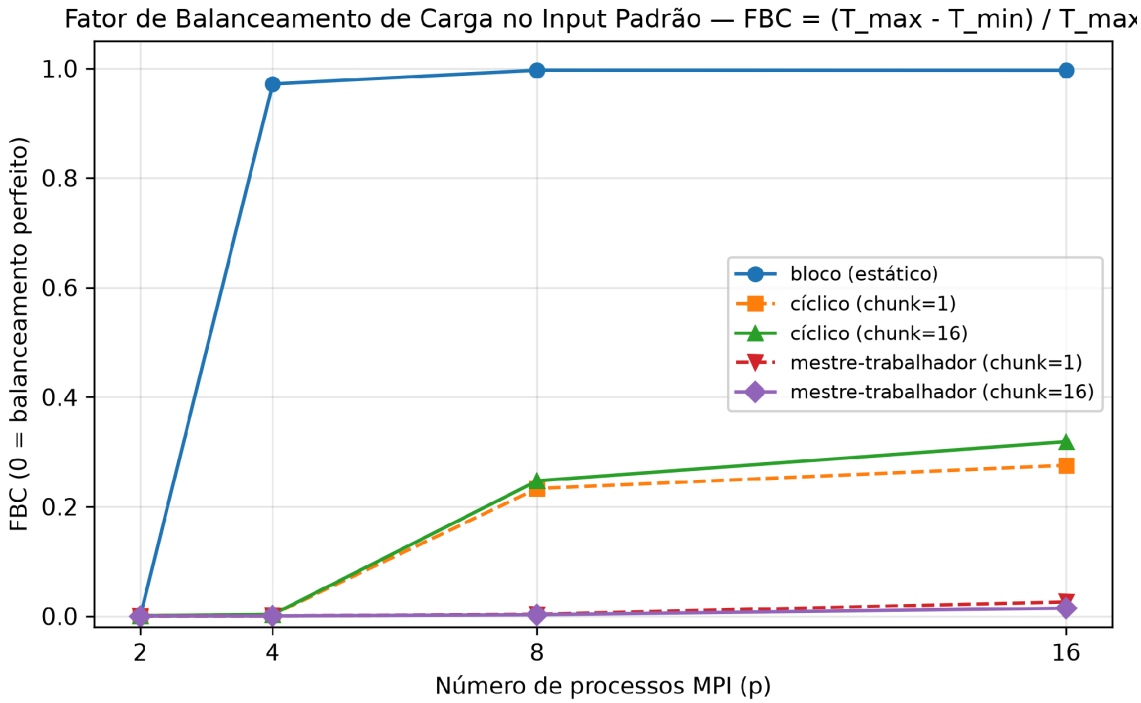
Tabela 4 - Fator de balanceamento de carga no input padrão

ESTRATÉGIA	P	T_MAX (S)	T_MIN (S)	F_LB(N)
bloco	2	2,8018	2,8005	0,0005
bloco	4	2,8671	0,0809	0,9718
bloco	8	2,2528	0,0066	0,9971
bloco	16	1,5639	0,0049	0,9969
ciclico, chunk 1	2	2,8009	2,7989	0,0007
ciclico, chunk 1	4	1,4778	1,4740	0,0026
ciclico, chunk 1	8	0,9771	0,7488	0,2331
ciclico, chunk 1	16	0,5827	0,4190	0,2755
ciclico, chunk 16	2	2,8016	2,7996	0,0007
ciclico, chunk 16	4	1,4767	1,4723	0,0030
ciclico, chunk 16	8	0,9795	0,7373	0,2472
ciclico, chunk 16	16	0,5771	0,3927	0,3185
mestre, chunk 1	2	5,5898	5,5898	0,0000
mestre, chunk 1	4	1,9620	1,9615	0,0003
mestre, chunk 1	8	0,9272	0,9242	0,0032
mestre, chunk 1	16	0,4867	0,4740	0,0262
mestre, chunk 16	2	5,5869	5,5869	0,0000
mestre, chunk 16	4	1,9768	1,9763	0,0003
mestre, chunk 16	8	0,9198	0,9176	0,0025
mestre, chunk 16	16	0,4976	0,4902	0,0145

Os números da estratégia bloco são o achado mais expressivo desta seção. Com quatro processos, o processo mais lento gasta 2,8671 s enquanto o mais rápido gasta apenas 0,0809 s, produzindo F_LB de 0,9718. Com dezesseis processos o fator chega a 0,9969, praticamente o valor máximo possível, o que significa que o primeiro processo a terminar fica ocioso durante quase toda a execução.

Há, porém, uma exceção reveladora. Com dois processos, a estratégia bloco registra F_LB de 0,0005, isto é, balanceamento praticamente perfeito. A explicação é geométrica. O Conjunto de Mandelbrot é simétrico em relação ao eixo real, e a região padrão está centrada nesse eixo. Ao dividir a imagem em exatamente duas faixas horizontais, cada processo recebe uma metade espelhada da outra, com carga idêntica. Essa coincidência desaparece assim que o número de faixas aumenta, pois as faixas centrais passam a concentrar o cardioide enquanto as faixas extremas ficam quase vazias.

Gráfico 4 - Fator de balanceamento de carga no input padrão



As estratégias cíclica e master-worker mantêm o fator muito mais baixo, porém não nulo. A estratégia cíclica apresenta fator de 0,2331 com oito processos e de 0,2755 com dezesseis, ainda que a quantidade de linhas atribuída a cada processo seja praticamente igual. Esse resíduo não decorre da geometria do fractal, e sim da arquitetura híbrida do processador, questão retomada na Seção 4.

3.2.3 Custo de Comunicação e Granularidade

O enunciado exige a análise do custo de coleta da imagem e, quando aplicável, da sobrecarga do coordenador. A tabela a seguir reúne as três medidas relevantes, o tempo acumulado em chamadas de MPI na raiz, o custo líquido de comunicação definido como T(n, p) − T_max, e a granularidade G(n) = T_cp / T_cm.

Tabela 5 - Custo de comunicação, granularidade e tempo de escrita no input padrão

ESTRATÉGIA	P	T EM MPI NA RAIZ (S)	T(N,P) – T_MAX (S)	G(N)	ESCRITA EM DISCO (S)
bloco	2	0,0115	0,0111	238,4	0,0342
bloco	4	2,8016	0,0158	1,0	0,0348
bloco	8	2,2570	0,0113	0,8	0,0354
bloco	16	1,5715	0,0126	0,7	0,0364
ciclico, chunk 1	2	0,0416	0,0424	88,4	0,0341
ciclico, chunk 1	4	0,0445	0,0422	64,5	0,0341
ciclico, chunk 1	8	0,2620	0,0362	5,9	0,0358
ciclico, chunk 1	16	0,1889	0,0336	4,9	0,0357
ciclico, chunk 16	2	0,0427	0,0429	87,7	0,0348
ciclico, chunk 16	4	0,0446	0,0448	58,4	0,0341
ciclico, chunk 16	8	0,1593	0,0390	4,8	0,0360
ciclico, chunk 16	16	0,1130	0,0401	4,2	0,0358
mestre, chunk 1	2	5,6013	0,0302	1,0	0,0347
mestre, chunk 1	4	1,9550	0,0120	3,0	0,0353
mestre, chunk 1	8	0,9173	0,0099	6,5	0,0369
mestre, chunk 1	16	0,4760	0,0119	10,5	0,0356
mestre, chunk 16	2	5,5916	0,0242	1,0	0,0347
mestre, chunk 16	4	1,9662	0,0099	3,0	0,0349
mestre, chunk 16	8	0,9071	0,0065	6,7	0,0362
mestre, chunk 16	16	0,4807	0,0075	11,8	0,0355

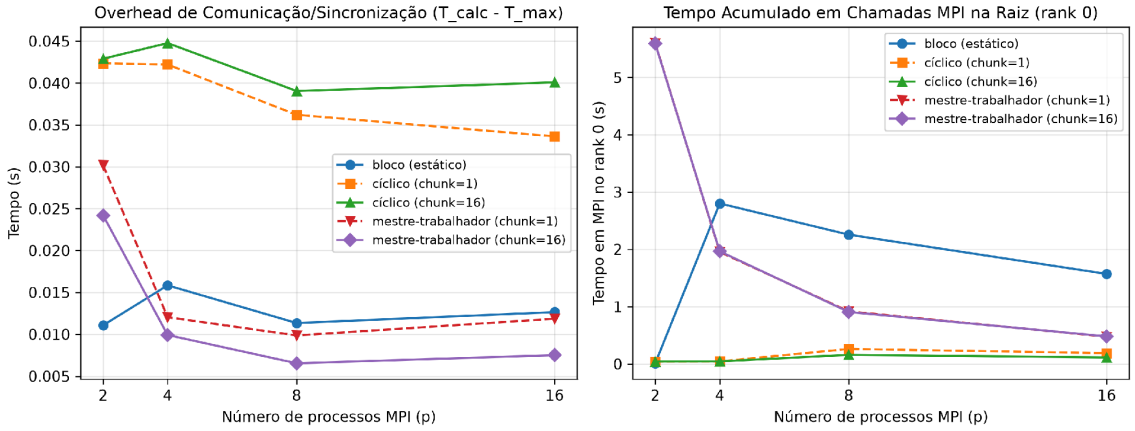
A leitura desta tabela exige o cuidado antecipado na Seção 2.5. O tempo acumulado em chamadas de MPI na raiz é alto justamente nas estratégias em que a raiz tem pouco ou nenhum trabalho útil. Na estratégia bloco com dezesseis processos esse valor é de 1,5715 s, praticamente igual ao tempo total de 1,5765 s, mas isso não significa que a coleta custou esse tempo. Significa que a raiz terminou a sua faixa quase

imediatamente e permaneceu bloqueada dentro de MPI_Gatherv aguardando os processos centrais.

O custo líquido de comunicação é a medida que isola o fenômeno. Ele permanece pequeno em todas as configurações do input padrão, variando de 0,0065 s a 0,0448 s, o que representa uma fração reduzida do tempo total. Isso confirma que, nesta escala e com todos os processos na mesma máquina, a transferência da matriz não é o gargalo. O tempo de escrita em disco, reportado à parte conforme exige o enunciado, também é pequeno e não entra em nenhuma métrica de speedup.

A granularidade reforça a mesma conclusão por outro ângulo. Nas estratégias estáticas com poucos processos, G(n) assume valores elevados, indicando granularidade grossa, com muito cálculo para pouca comunicação. À medida que o número de processos cresce, G(n) diminui, e na estratégia bloco aproxima-se da unidade. É preciso destacar que essa queda não reflete aumento de tráfego de mensagens, e sim o crescimento do tempo ocioso embutido em T_cm, uma vez que os processos rápidos passam a esperar cada vez mais pelos lentos.

Gráfico 5 - Custo de comunicação e tempo acumulado em chamadas de MPI na raiz



3.3 Caso de Desbalanceamento Acentuado

Esta seção trata do cenário de estresse, que usa a região do vale dos cavalos-marinhos centrada em -0,743643887 e 0,131825904, com largura real de $3{,}0 \times 10^{-3}$ e MAX_ITER = 5000. Nessa região a concentração de pixels de custo elevado é extrema, e a análise do fator de balanceamento de carga torna-se central para avaliar as soluções paralelas.

Tabela 6 - Caso de zoom, tempo de execução, speedup, eficiência e balanceamento

ESTRATÉGIA	P	T(N,P) (S)	S(N,P)	E(N,P)	F_LB(N)
bloco	2	1,5373	1,59	0,797	0,3380
bloco	4	0,8295	2,95	0,738	0,4889
bloco	8	0,5736	4,27	0,534	0,7312
bloco	16	0,3400	7,21	0,450	0,6951
ciclico, chunk 1	2	1,3241	1,85	0,925	0,0119
ciclico, chunk 1	4	0,6927	3,54	0,884	0,0042
ciclico, chunk 1	8	0,4716	5,20	0,649	0,2399
ciclico, chunk 1	16	0,3072	7,98	0,498	0,2667
ciclico, chunk 16	2	1,3075	1,87	0,937	0,0090
ciclico, chunk 16	4	0,6872	3,57	0,891	0,0105
ciclico, chunk 16	8	0,4759	5,15	0,644	0,2513
ciclico, chunk 16	16	0,2776	8,83	0,552	0,2180
mestre, chunk 1	2	2,5491	0,96	0,481	0,0000
mestre, chunk 1	4	0,8738	2,80	0,701	0,0010
mestre, chunk 1	8	0,4178	5,86	0,733	0,0083
mestre, chunk 1	16	0,2237	10,95	0,685	0,1740
mestre, chunk 16	2	2,5458	0,96	0,481	0,0000
mestre, chunk 16	4	0,8831	2,77	0,694	0,0127
mestre, chunk 16	8	0,4115	5,95	0,744	0,0465
mestre, chunk 16	16	0,2258	10,85	0,678	0,0853

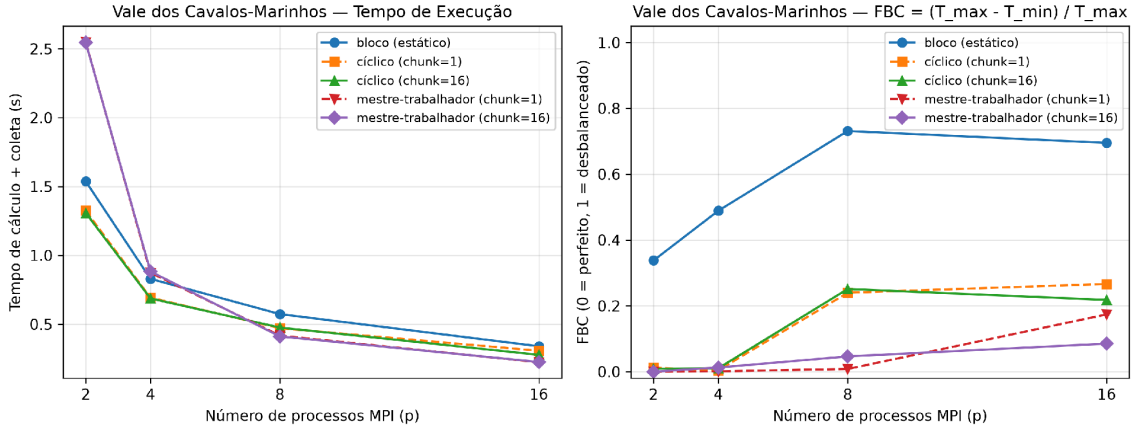
O comportamento do caso de zoom difere do input padrão em um aspecto importante. A estratégia bloco, que no input padrão apresentava balanceamento quase perfeito com dois processos por efeito da simetria do conjunto, aqui registra F_LB de 0,3380 já com dois processos. A região ampliada não está centrada no eixo real de forma simétrica, e por isso a coincidência favorável desaparece. Com dezesseis processos o fator atinge 0,6951.

Ainda assim, a estratégia bloco apresenta no zoom um desempenho relativamente melhor do que no input padrão, com speedup de 7,21 contra 3,59 no caso padrão. A

razão é que, no vale dos cavalos-marinhos, os pixels de custo elevado estão distribuídos ao longo de filamentos que atravessam várias faixas horizontais, em vez de se concentrarem em um bloco central compacto como ocorre com o cardioide principal.

O esquema master-worker novamente se destaca, atingindo speedup de 10,95 com dezesseis processos e fator de 0,1740. A estratégia cíclica com chunk 16 fica em 8,83. Também aqui o master-worker com dois processos registra speedup de 0,96, abaixo da unidade, configurando a situação de slowdown apontada em sala de aula. Com um único trabalhador, o esquema executa o trabalho sequencial inteiro e ainda paga o custo de empacotar e transmitir cada bloco de linhas ao coordenador.

Gráfico 6 - Vale dos cavalos-marinhos, tempo de execução e fator de balanceamento



3.4 Escalabilidade Fraca

Na escalabilidade fraca, a resolução cresce junto com o número de processos, mantendo o trabalho por processo aproximadamente constante, segundo $N = 4096 \cdot \sqrt{p}$. Conforme o material da disciplina, um programa é escalável se for possível manter constante a eficiência $E(n, p)$ ao incrementar o tamanho do problema n ao mesmo tempo que se incrementa o número de processos p. Por esse motivo, a eficiência apresentada nesta seção utiliza a mesma fórmula das seções anteriores, com o baseline sequencial medido na própria resolução de cada ponto.

Tabela 7 - Escalabilidade fraca, resolução, baseline sequencial, tempo, speedup e eficiência

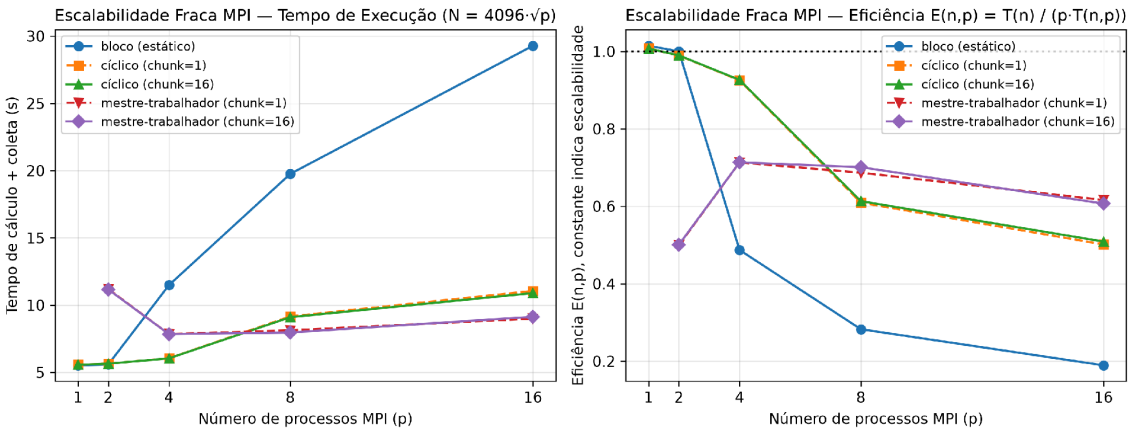
ESTRATÉGIA	P	RESOLUÇÃO	T(N) (S)	T(N,P) (S)	S(N,P)	E(N,P)
bloco	1	4096²	5,608	5,5257	1,01	1,015
bloco	2	5793²	11,184	5,5910	2,00	1,000
bloco	4	8192²	22,392	11,4893	1,95	0,487
bloco	8	11585²	44,670	19,7588	2,26	0,283
bloco	16	16384²	88,735	29,2854	3,03	0,189
ciclico, chunk 1	1	4096²	5,608	5,5641	1,01	1,008
ciclico, chunk 1	2	5793²	11,184	5,6518	1,98	0,989
ciclico, chunk 1	4	8192²	22,392	6,0486	3,70	0,925
ciclico, chunk 1	8	11585²	44,670	9,1607	4,88	0,610
ciclico, chunk 1	16	16384²	88,735	11,0569	8,03	0,502
ciclico, chunk 16	1	4096²	5,608	5,5654	1,01	1,008
ciclico, chunk 16	2	5793²	11,184	5,6486	1,98	0,990
ciclico, chunk 16	4	8192²	22,392	6,0415	3,71	0,927
ciclico, chunk 16	8	11585²	44,670	9,1029	4,91	0,613
ciclico, chunk 16	16	16384²	88,735	10,8965	8,14	0,509
mestre, chunk 1	2	5793²	11,184	11,1949	1,00	0,500
mestre, chunk 1	4	8192²	22,392	7,8468	2,85	0,713
mestre, chunk 1	8	11585²	44,670	8,1291	5,50	0,687
mestre, chunk 1	16	16384²	88,735	8,9987	9,86	0,616
mestre, chunk 16	2	5793²	11,184	11,1620	1,00	0,501
mestre, chunk 16	4	8192²	22,392	7,8450	2,85	0,714
mestre, chunk 16	8	11585²	44,670	7,9638	5,61	0,701
mestre, chunk 16	16	16384²	88,735	9,1364	9,71	0,607

A estratégia bloco apresenta a degradação mais acentuada, com eficiência caindo de 1,015 com um processo para 0,189 com dezesseis. A estratégia cíclica com chunk 16 parte de 1,008 e termina em 0,509. O esquema master-worker com chunk 16, depois de absorver a penalidade estrutural do coordenador com dois processos, que o coloca em 0,501, sobe para 0,714 com quatro processos e encerra em 0,607 com dezesseis.

Nenhuma das estratégias mantém a eficiência constante ao longo de toda a faixa, de modo que, pelo critério adotado em sala de aula, o programa não é plenamente escalável neste hardware. É importante observar, contudo, que a curva do master-worker é a mais estável a partir de quatro processos, variando pouco entre quatro e dezesseis processos, ao passo que a estratégia bloco cai de forma monotônica e acentuada. Esse contraste indica que boa parte da perda de escalabilidade observada tem origem na distribuição do trabalho, e não apenas em limitações de hardware.

Dois efeitos adicionais aparecem nos pontos de maior resolução. O primeiro é o aumento do desvio padrão, que na estratégia bloco com dezesseis processos chega a 5,2428 s sobre um tempo médio de 29,2854 s. O segundo é o crescimento do custo líquido de comunicação, que nas resoluções maiores passa a transferir matrizes da ordem de centenas de megabytes, chegando a 0,8129 s. Ambos são discutidos na *Seção 4*.

Gráfico 7 - Escalabilidade fraca, tempo de execução e eficiência E(n, p)



3.5 Comparação com a Solução OpenMP da Etapa 1

O enunciado exige a comparação entre a solução em memória distribuída e a solução em memória compartilhada desenvolvida na Etapa 1. A comparação apresentada aqui é possível porque as duas etapas foram medidas na mesma máquina, com o mesmo compilador, com o mesmo núcleo de cálculo e com o governador de frequência fixado em performance nas duas ocasiões. Os baselines sequenciais das duas coletas diferem em menos de 2%, o que confirma a equivalência das condições.

Tabela 8 - Comparação de speedup entre MPI e OpenMP no input padrão, por estratégia equivalente

ESTRATÉGIA EQUIVALENTE	P	SPEEDUP MPI	SPEEDUP OPENMP	RAZÃO MPI/OPENMP
Decomposição contígua	2	2,01	1,93	1,04
Decomposição contígua	4	1,96	2,02	0,97
Decomposição contígua	8	2,50	2,85	0,88
Decomposição contígua	16	3,59	4,57	0,79
Distribuição cíclica com chunk 16	2	1,99	1,97	1,01
Distribuição cíclica com chunk 16	4	3,72	3,74	0,99
Distribuição cíclica com chunk 16	8	5,56	6,04	0,92
Distribuição cíclica com chunk 16	16	9,17	8,45	1,09
Balanceamento dinâmico com chunk 16	2	1,01	1,98	0,51
Balanceamento dinâmico com chunk 16	4	2,85	3,75	0,76
Balanceamento dinâmico com chunk 16	8	6,11	6,81	0,90
Balanceamento dinâmico com chunk 16	16	11,20	10,44	1,07

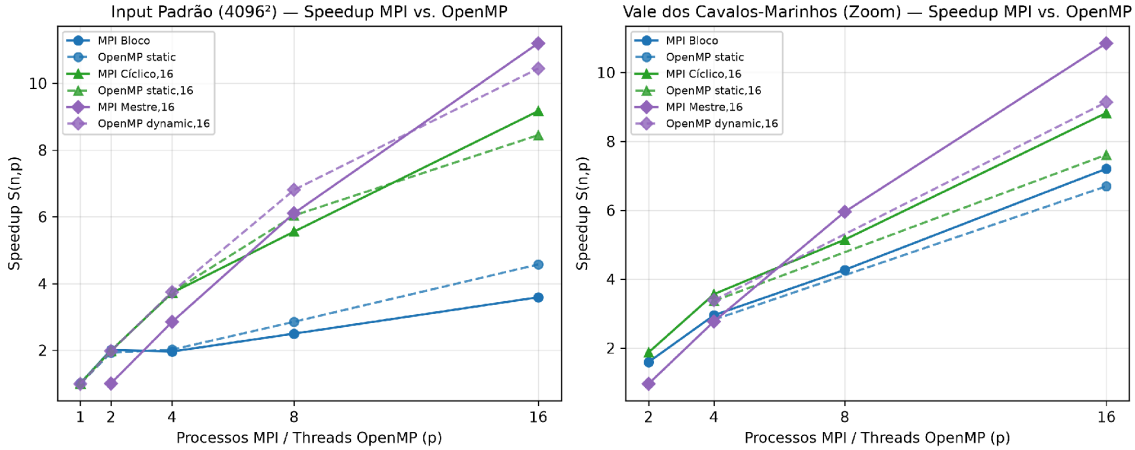
A comparação revela um padrão consistente. Nas configurações bem balanceadas, a solução MPI alcança speedup comparável ao da solução OpenMP, e no caso do balanceamento dinâmico chega a superá-la com dezesseis processos. Nas configurações mal balanceadas, representadas pela decomposição contígua, a solução MPI fica atrás da equivalente em OpenMP.

Esse último resultado tem explicação direta. Em OpenMP, mesmo com particionamento contíguo, as threads compartilham a matriz de saída e não existe etapa de coleta, de modo que uma thread que termina cedo apenas encerra. Em MPI, o processo que termina cedo precisa permanecer bloqueado em MPI_Gatherv até que o processo mais lento chegue à mesma chamada coletiva, e só então os dados são transferidos. O

desbalanceamento, portanto, cobra um preço adicional em memória distribuída, pois converte tempo ocioso em tempo de sincronização obrigatória.

Quanto ao balanceamento, as duas etapas chegam à mesma ordenação qualitativa das estratégias. Tanto em OpenMP quanto em MPI, o particionamento contíguo produz o pior balanceamento, a distribuição com granularidade fina melhora substancialmente o quadro, e o balanceamento dinâmico produz o melhor resultado. Como registrado na *Seção 2.5*, os valores numéricos do fator não são comparáveis entre as etapas, porque as definições adotadas são diferentes, e por isso a comparação se restringe aqui à ordenação relativa.

Gráfico 8 - Comparação de speedup entre MPI e OpenMP no input padrão e no caso de zoom



No Gráfico 8, as curvas de OpenMP do caso de zoom aparecem com menos pontos do que as de MPI. Isso ocorre porque a Etapa 1 coletou o cenário de zoom apenas com 4 e 16 threads, ao passo que esta etapa cobriu 2, 4, 8 e 16 processos.

3.6 Síntese dos Resultados

A tabela a seguir consolida as melhores e piores configurações de cada cenário, que sustentam a discussão apresentada na Seção 4:

Tabela 9 - Síntese dos resultados obtidos com 16 processos

CENÁRIO	MELHOR CONFIGURAÇÃO	SPEEDUP	PIOR CONFIGURAÇÃO	SPEEDUP
Input padrão	mestre, chunk 1	11,35	bloco	3,59
Zoom, cavalos-marinhos	mestre, chunk 1	10,95	bloco	7,21
Escalabilidade fraca	mestre, chunk 1	9,86	bloco	3,03

Em síntese, os resultados convergem para quatro conclusões centrais. Primeiro, a corretude foi preservada integralmente, e a saída das três estratégias é bit a bit idêntica à da referência sequencial nos dois inputs. Segundo, o esquema master-worker foi a estratégia mais consistente nos dois cenários de escalabilidade forte, apesar de sacrificar um processo inteiro para a coordenação. Terceiro, a decomposição contígua colapsa de forma severa, com fator de balanceamento próximo do máximo teórico a partir de quatro processos. Quarto, o custo líquido de comunicação permaneceu pequeno em todos os cenários do input padrão, o que indica que, nesta escala e com todos os processos em uma única máquina, o gargalo dominante é a distribuição do trabalho, e não a troca de mensagens.

4. Discussão

Esta seção interpreta os resultados da *Seção 3* à luz dos conceitos teóricos da Introdução e das decisões metodológicas registradas na *Seção 2*. A discussão está organizada em torno de nove eixos: a corretude, a comparação entre a versão sequencial e as paralelas, a resposta de cada estratégia ao desbalanceamento, o custo estrutural do coordenador, o custo de comunicação e a granularidade, os limites de Amdahl e da arquitetura, a escalabilidade fraca e Gustafson, a comparação com OpenMP e, por fim, os demais gargalos e as sugestões de melhoria.

4.1 Análise da Corretude

A corretude foi validada por comparação bit a bit entre a matriz de contagens produzida por cada estratégia e a da versão sequencial de referência, nos dois inputs. Em todos os seis cenários o resultado foi de igualdade exata, sem nenhum pixel divergente.

Esse desfecho é consequência estrutural do desenho da paralelização. O Mandelbrot por *escape-time* é embaraçosamente paralelo, cada pixel é computado de forma independente, e a função *escape_time* executa exatamente a mesma sequência de operações em ponto flutuante qualquer que seja o processo que a execute. Como a

atribuição de trabalho não altera a ordem das operações dentro de cada pixel, não há reassociação capaz de produzir divergência de fronteira.

Em memória distribuída há um risco adicional que não existia na Etapa 1 e que merece registro. Além de calcular corretamente, é preciso remontar a matriz na ordem certa, pois cada processo devolve apenas um subconjunto das linhas. Na estratégia cíclica, as linhas de um mesmo processo não são contíguas na imagem final, o que exige uma etapa explícita de reordenação na raiz. Um erro nessa remontagem não produziria valores numericamente errados, e sim uma imagem com linhas embaralhadas, que passaria despercebida em uma inspeção superficial do tempo de execução. A validação bit a bit cobre também esse risco, e o fato de a estratégia cíclica produzir saída idêntica confirma que o mapeamento de índices está correto.

4.2 Comparação Entre a Versão Sequencial e as Versões Paralelas

Com dezesseis processos e o baseline sequencial de 5,658 s no input padrão, os resultados variam de um speedup de 3,59 na decomposição contígua até 11,35 no esquema master-worker com chunk 1. Em outras palavras, para o mesmo problema, o mesmo hardware e o mesmo núcleo de cálculo, a escolha da estratégia de distribuição mais do que triplica o ganho obtido.

Esse contraste é a lição central da etapa e reforça o que a Etapa 1 já havia indicado em memória compartilhada. Paralelizar, por si só, não garante desempenho. Como as versões paralelas são estruturalmente idênticas à sequencial no núcleo de cálculo, toda a diferença observada decorre exclusivamente da forma como o trabalho foi dividido e de como o resultado foi coletado.

4.3 Por Que o Problema é Desbalanceado e a Resposta de Cada Estratégia

O tempo necessário para processar cada ponto é proporcional ao número de iterações do laço de escape. A vasta maioria dos pixels situados em regiões afastadas do fractal escapa rapidamente do raio pré-definido, exigindo pouco processamento, enquanto os pixels no interior ou na fronteira do conjunto demandam as 1000 iterações completas no input padrão, e até 5000 iterações no caso de zoom. Como essa disparidade é geograficamente concentrada, qualquer divisão que respeite a vizinhança espacial herda a concentração.

A estratégia bloco é a tradução direta dessa armadilha. Ao atribuir faixas contíguas de linhas, os processos que recebem a região central herdam o cardioide principal, ao passo que os processos das extremidades recebem regiões quase vazias. Com quatro processos o fator de balanceamento salta para 0,9718, e com dezesseis chega a 0,9969. O processo mais rápido conclui a sua faixa em 0,0049 s, enquanto o mais lento gasta 1,5639 s, e permanece ocioso durante praticamente toda a execução.

O caso de dois processos merece atenção especial, porque é um resultado que poderia ser lido erroneamente como evidência de que a decomposição contígua funciona bem. Com p = 2 o fator é de apenas 0,0005, praticamente ideal. Esse valor, contudo, não decorre de mérito da estratégia, e sim da simetria do Conjunto de Mandelbrot em relação ao eixo real combinada com o fato de a região padrão estar centrada nesse eixo. Ao dividir a imagem em duas faixas horizontais, cada processo recebe uma metade espelhada da outra. A coincidência é destruída assim que o número de faixas aumenta, e desaparece por completo no caso de zoom, em que a região ampliada não é simétrica e o fator com dois processos já é de 0,3380.

As demais estratégias atacam o problema por caminhos distintos, com resultados que convém comparar diretamente:

- **Distribuição cíclica:** ao distribuir blocos de linhas em round-robin, a estratégia quebra a contiguidade espacial e faz com que cada processo receba amostras de todas as regiões da imagem, tanto das caras quanto das baratas. O resultado é uma queda expressiva do fator, que com dezesseis processos fica em 0,2755 com chunk 1 e em 0,3185 com chunk 16, contra um valor próximo da unidade na decomposição contígua. Tudo isso sem nenhuma negociação em tempo de execução, pois a atribuição continua sendo calculada por fórmula, com custo de escalonamento nulo.
- **Master-worker:** ao entregar tarefas sob demanda, o esquema adapta a distribuição ao custo real de cada bloco, em vez de supor que a amostragem cíclica será suficiente. Com dezesseis processos o fator cai para 0,0262 com chunk 1 e para 0,0145 com chunk 16, os dois menores valores entre todas as configurações com esse número de processos no input padrão. O preço é o tráfego de mensagens proporcional ao número de tarefas e a dedicação integral de um processo à coordenação.

Chama atenção que, no input padrão, a diferença entre chunk 1 e chunk 16 seja pequena nas duas estratégias dinâmicas. Isso difere do observado na Etapa 1, em que a granularidade fina foi decisiva. A explicação está no custo relativo. Em memória compartilhada, reduzir o chunk custa apenas sincronização em uma fila local. Em memória distribuída, reduzir o chunk multiplica o número de mensagens, e esse custo compensa parte do ganho de balanceamento. O resultado é que, em MPI, o ponto ótimo de granularidade é mais grosso do que em OpenMP.

4.4 O Custo Estrutural do Coordenador no Master-Worker

O esquema master-worker implementado dedica o rank 0 exclusivamente à coordenação, de modo que com p processos apenas p − 1 produzem trabalho útil. Essa decisão tem uma consequência direta sobre as métricas que precisa ser explicitada, sob pena de os números serem mal interpretados.

A fórmula de eficiência divide o speedup pelo número de processos *p*, e não pelo número de processos que calculam. Com dois processos, o master-worker registra eficiência de 0,503 no input padrão, um valor que parece indicar falha grave, mas que é simplesmente o teto estrutural da configuração. Com um único trabalhador, o tempo paralelo não pode ser inferior ao sequencial, e o speedup máximo alcançável é 1, o que produz eficiência de 0,5 por definição.

No caso de zoom a situação é ainda mais clara, pois o speedup com dois processos é de 0,96, abaixo da unidade. Trata-se exatamente da situação de slowdown, em que a sobrecarga da paralelização supera o ganho. O único trabalhador executa todo o trabalho sequencial e ainda paga o custo de empacotar cada bloco de linhas e transmiti-lo ao coordenador.

A penalidade se dilui rapidamente conforme *p* cresce, pois a fração de processos desperdiçada é 1/p. Com quatro processos a eficiência já sobe para 0,717, atinge o seu máximo de 0,755 com oito processos e recua levemente para 0,709 com dezesseis, ponto em que ainda assim se registra o maior speedup de toda a etapa, de 11,35. Ou seja, o investimento de um processo em coordenação só compensa quando há trabalhadores suficientes para que o balanceamento dinâmico recupere mais do que o processo sacrificado custou. Neste experimento, o ponto de virada situa-se entre dois e quatro processos, e o recuo final da eficiência reflete os limites arquiteturais discutidos na *Seção 4.6*.

Uma alternativa de projeto seria fazer o coordenador também calcular nos intervalos entre atendimentos, o que eliminaria o desperdício. Essa variante não foi implementada porque exigiria recepção não bloqueante para evitar que um trabalhador ficasse esperando enquanto o coordenador calculasse, o que aumentaria a complexidade sem alterar a conclusão principal sobre balanceamento. Ela fica registrada entre as sugestões de melhoria.

4.5 Custo de Comunicação e Granularidade

A separação entre tempo de comunicação e tempo ocioso, antecipada na *Seção 2.5*, é o que permite responder com precisão se a troca de mensagens foi ou não um gargalo nesta etapa. A resposta é que não foi.

O tempo acumulado em chamadas de MPI na raiz é elevado justamente nas configurações em que a raiz tem pouco trabalho útil, e isso é um artefato de medição, não um custo real de transferência. Na estratégia bloco com dezesseis processos, esse tempo é de 1,5715 s sobre um tempo total de 1,5765 s. A raiz recebe a primeira faixa da imagem, que é quase vazia, conclui o seu cálculo quase imediatamente e permanece bloqueada dentro de MPI_Gatherv até que os processos centrais terminem. O que esse número mede é ociosidade imposta pelo desbalanceamento, e não largura de banda.

O custo líquido de comunicação, definido como T(n, p) − T_max, isola o fenômeno porque desconta o trabalho útil do processo mais lento. No input padrão esse custo permanece entre 0,0065 s e 0,0448 s em todas as configurações, o que representa uma fração pequena do tempo total mesmo nas execuções mais rápidas. A conclusão é que, nesta escala e com todos os processos executando na mesma máquina física, a transferência da matriz pela memória do sistema é barata, e o gargalo dominante é a distribuição do trabalho.

A granularidade G(n) = T_cp / T_cm confirma a mesma leitura e, ao mesmo tempo, ilustra por que a métrica precisa ser interpretada com cuidado neste problema. Nas estratégias estáticas com poucos processos, G(n) assume valores de várias dezenas ou centenas, indicando granularidade grossa, com muito cálculo para pouca comunicação, exatamente o regime desejável. Conforme *p* cresce, G(n) diminui, e na estratégia bloco aproxima-se da unidade. Como o denominador T_cm agrega comunicação e espera, essa queda não indica aumento de tráfego, e sim que o tempo ocioso passou a rivalizar com o tempo de cálculo. A granularidade, nesse caso, está medindo o colapso do balanceamento por via indireta.

Cabe ainda registrar o efeito das estratégias sobre o custo de coleta. A estratégia bloco permite que a raiz escreva diretamente na matriz final, usando MPI_IN_PLACE, sem cópia intermediária. A estratégia cíclica, por distribuir linhas não contíguas, exige um buffer intermediário e uma etapa de reordenação na raiz, custo que foi deliberadamente contabilizado dentro do tempo de comunicação para não favorecer artificialmente essa estratégia. O master-worker dispensa a coleta coletiva, pois cada bloco já chega acompanhado da sua posição de destino e é copiado diretamente para o lugar certo.

4.6 Escalabilidade Forte, Lei de Amdahl e Limites Arquiteturais

A análise da escalabilidade forte revela dois comportamentos distintos conforme a estratégia. Para as estratégias bem balanceadas, o tempo decai de forma aproximadamente proporcional até quatro processos, com eficiência de 0,931 na estratégia cíclica, o que está próximo do ideal nessa faixa. A partir de oito processos a degradação se acentua, caindo para 0,698 e depois para 0,574 com dezesseis processos.

O primeiro fator que explica essa limitação é inerente à Lei de Amdahl. O speedup de um programa paralelo é limitado por S_M = 1 / (1 − f), onde f é a fração perfeitamente paralelizável. Embora a fase de cálculo do *escape-time* seja altamente paralelizável, existem parcelas inerentemente sequenciais nesta implementação, a saber a inicialização do ambiente MPI, a alocação da matriz completa na raiz, a reordenação das linhas na estratégia cíclica e a própria coleta coletiva, que não pode concluir antes que o último processo chegue. Essas parcelas estabelecem o teto superior observado.

O segundo fator, igualmente importante, é a natureza híbrida do processador utilizado. O Intel Core 7 240H possui 10 núcleos físicos e 16 CPUs lógicas, combinando P-cores

com Hyper-Threading e E-cores sem Hyper-Threading. Isso significa que, ao escalar de 8 para 16 processos, a execução passa a utilizar simultaneamente threads lógicas que disputam as mesmas unidades de hardware nos P-cores e núcleos E, que operam de forma mais lenta. Não existem, portanto, dezesseis unidades de execução equivalentes, e a premissa registrada em aula de que cada processo é executado em um processador físico diferente deixa de valer nessa faixa.

Essa assimetria de hardware tem uma manifestação direta e mensurável nos resultados, que é o resíduo do fator de balanceamento nas estratégias estáticas bem distribuídas. A estratégia cíclica atribui a cada processo praticamente o mesmo número de linhas, amostradas de todas as regiões da imagem, e ainda assim registra fator de 0,2755 com dezesseis processos. Como a quantidade de trabalho é equivalente, a diferença de tempo só pode vir da velocidade do núcleo em que cada processo foi alocado. Trata-se de desbalanceamento induzido pelo hardware, e não pela geometria do fractal.

Esse diagnóstico explica também por que o esquema master-worker apresenta fator substancialmente menor no mesmo ponto, de 0,0262. O balanceamento dinâmico opera em dois níveis simultaneamente. Ele dilui a heterogeneidade geométrica do fractal, que era o objetivo original, e compensa de forma automática a heterogeneidade do hardware, pois um processo alocado em um E-core simplesmente solicita menos tarefas ao longo da execução. Nenhuma estratégia estática consegue reproduzir esse segundo efeito, porque a atribuição é decidida antes de a execução começar.

4.7 Escalabilidade Fraca e a Lei de Gustafson

Os materiais da disciplina definem escalabilidade como a capacidade de manter constante a eficiência E(n, p) ao incrementar o tamanho do problema *n* junto com o número de processos *p*, abordagem proposta por Gustafson como resposta à principal deficiência da formulação de Amdahl, que é não considerar o tamanho do problema. O raciocínio é que, ao aumentar *n*, a fração inerentemente sequencial 1 − f torna-se consideravelmente menor.

Os resultados mostram que esse mecanismo opera apenas parcialmente neste experimento, e de forma muito desigual entre as estratégias. A decomposição contígua cai de eficiência 1,015 com um processo para 0,189 com dezesseis, uma degradação severa. A estratégia cíclica com chunk 16 vai de 1,008 a 0,509. O master-worker com chunk 16, depois de superar a penalidade do coordenador, mantém-se entre 0,714 e 0,607 na faixa de quatro a dezesseis processos, que é a curva mais estável das três.

A leitura conjunta com a *Seção 4.6* é esclarecedora. Se a perda de escalabilidade fraca decorresse apenas das limitações de hardware identificadas na escalabilidade forte, todas as estratégias deveriam degradar de modo semelhante, pois todas usam o mesmo processador e a mesma hierarquia de memória. O que se observa, porém, é uma dispersão grande entre elas no mesmo ponto. Isso indica que a parcela dominante

da perda tem origem na distribuição do trabalho, e que a limitação arquitetural atua como um piso adicional, visível na curva do master-worker, que é a menos afetada pelo desbalanceamento.

Dois efeitos próprios da memória distribuída aparecem nos pontos de maior resolução e não tinham equivalente na Etapa 1. O primeiro é o crescimento do custo líquido de comunicação, que atinge 0,8129 s nas resoluções maiores. Em 16384 × 16384, a matriz de contagens ocupa 1 GiB, e nas estratégias que exigem buffer intermediário a raiz precisa alocar o dobro disso, somando a reordenação das linhas. O segundo é o aumento do desvio padrão entre repetições, que na estratégia bloco com dezesseis processos chega a 5,2428 s sobre uma média de 29,2854 s. Essa variabilidade é compatível com pressão sobre a hierarquia de memória, pois as matrizes maiores deixam de caber no cache L3 de 24 MiB e passam a depender da largura de banda da DRAM, que é compartilhada por todos os processos.

4.8 Comparação com a Solução OpenMP

A comparação entre os dois paradigmas, apresentada na *Seção 3.5*, permite avaliar vantagens e custos de cada um neste problema específico. É importante frisar que a comparação só é legítima porque as duas coletas foram feitas na mesma máquina, com o mesmo compilador, o mesmo núcleo de cálculo e o governador de frequência fixado em performance, com baselines sequenciais que diferem em menos de 2%.

Nas configurações bem balanceadas, as duas soluções apresentam desempenho comparável, e o balanceamento dinâmico em MPI chega a superar o equivalente em OpenMP com dezesseis processos. Esse resultado é relevante porque contraria a expectativa intuitiva de que memória distribuída deveria ser sempre mais lenta em uma única máquina. A explicação está no fato de que, neste problema, cada processo escreve exclusivamente na sua própria região de memória, sem nenhum compartilhamento de escrita. Em OpenMP, threads que escrevem em linhas vizinhas da mesma matriz podem sofrer falso compartilhamento de linhas de cache, efeito que não existe quando cada processo possui o seu próprio buffer.

Nas configurações mal balanceadas o quadro se inverte, e a solução MPI fica atrás da equivalente em OpenMP. A razão é estrutural. Em memória compartilhada, uma thread que termina cedo apenas encerra, e o custo do desbalanceamento é o tempo ocioso. Em memória distribuída, o processo que termina cedo precisa permanecer bloqueado em uma chamada coletiva até que o último processo chegue, e só então os dados são efetivamente transferidos. O desbalanceamento, portanto, não apenas desperdiça tempo, como também atrasa o início da própria coleta, acumulando dois custos em sequência.

Em termos de complexidade de implementação, a diferença é considerável. A versão OpenMP da Etapa 1 obteve as três políticas de escalonamento alterando essencialmente uma diretiva e uma variável de ambiente. A versão MPI exigiu

implementar explicitamente a decomposição do domínio, o cálculo dos vetores de contagens e deslocamentos para a coleta, a reordenação das linhas na estratégia cíclica e um protocolo completo de requisição e entrega de tarefas no master-worker. Em contrapartida, a solução MPI oferece algo que a solução OpenMP não oferece, que é a possibilidade de ultrapassar os limites de uma única máquina, distribuindo o cálculo por vários nós de um cluster.

4.9 Outros Gargalos e Limitações

Além das limitações já discutidas, três pontos merecem registro explícito como limitações deste estudo:

- **Execução em nó único:** todos os processos MPI foram executados em uma única máquina física, de modo que a comunicação ocorre pela memória do sistema e não por uma rede real. Os custos de comunicação medidos aqui são, portanto, um limite inferior. Em um cluster com interconexão de rede, a latência e a largura de banda alterariam o equilíbrio entre as estratégias, e muito provavelmente penalizariam a granularidade fina, que gera mais mensagens.
- **Oversubscribe acima de dez processos:** com dezesseis processos em dez núcleos físicos, foi necessário recorrer a oversubscribe, e o sistema operacional passou a mediar o acesso às unidades de execução. Isso introduz uma fonte de variabilidade que não existe quando há um processo por núcleo físico, e explica parte do desvio padrão observado nos pontos de dezesseis processos.
- **Agregação entre comunicação e espera:** a medida de tempo em chamadas de MPI não separa, por construção, o tempo de transferência do tempo de espera. Essa limitação foi contornada com a introdução do custo líquido de comunicação, porém uma separação completa exigiria instrumentação adicional com chamadas não bloqueantes ou uso de ferramentas de profiling específicas de MPI.

Cabe ainda registrar, por rigor metodológico, que a coleta preliminar conduzida com o governador em powersave produziu tempos sequenciais em torno de 37% superiores aos da coleta final. Esse episódio confirma, agora em um segundo paradigma, a conclusão já registrada na Etapa 1 de que a fixação da frequência é pré-requisito de confiabilidade das medições, e não um detalhe de configuração.

4.10 Sugestões de Melhorias Futuras

A partir dos resultados obtidos, identificam-se quatro direções de aprofundamento, todas restritas ao escopo do estudo do paralelismo:

- **Coordenador produtivo no master-worker:** permitir que o rank 0 execute tarefas nos intervalos entre atendimentos, utilizando MPI_Irecv e verificação não

bloqueante, eliminaria o desperdício de um processo inteiro. O ganho esperado é maior justamente onde o esquema hoje é mais penalizado, que é a faixa de poucos processos.

- **Sobreposição entre comunicação e computação:** substituir a coleta coletiva por envios não bloqueantes, permitindo que a transferência de uma faixa já concluída ocorra enquanto outras faixas ainda são calculadas. Os materiais didáticos da disciplina listam explicitamente a sobreposição entre comunicação e computação entre os fatores a considerar para obter algoritmos paralelos eficientes, e esta implementação não a explora.
- **Varredura sistemática de granularidade:** avaliar chunks de 1, 2, 4, 8, 16, 32 e 64 de forma sistemática permitiria traçar a curva exata do compromisso entre diluição da carga e número de mensagens, e localizar com precisão o ponto ótimo em memória distribuída, que os resultados indicam ser mais grosso do que em memória compartilhada.
- **Controle de afinidade de processos:** repetir a bateria com vinculação explícita de processos a núcleos, separando execuções restritas a P-cores de execuções restritas a E-cores, permitiria quantificar com exatidão quanto do fator de balanceamento residual observado nas estratégias estáticas decorre da assimetria do hardware e quanto decorre da geometria do problema.

Por fim, a extensão natural deste estudo é a execução em um cluster com mais de um nó, único cenário em que o custo real de rede se manifesta e em que a vantagem estrutural do paradigma de memória distribuída sobre o de memória compartilhada pode ser efetivamente demonstrada.

5. Conclusão

A Etapa 2 do projeto cumpriu o objetivo proposto, que era implementar, validar e comparar experimentalmente estratégias de distribuição do domínio da imagem em memória distribuída com MPI, isolando o efeito dessas estratégias sobre o desbalanceamento de carga inerente ao Conjunto de Mandelbrot. A corretude foi preservada em todos os cenários testados, e a matriz de contagens das três estratégias foi bit a bit idêntica à referência sequencial tanto no input padrão quanto no caso de zoom.

O principal aprendizado desta etapa não foi que a paralelização em memória distribuída reduz o tempo de execução, o que já era esperado, mas sim que o custo do desbalanceamento muda de natureza quando se sai da memória compartilhada. Em OpenMP, uma thread mal servida apenas termina cedo e fica ociosa. Em MPI, o processo que termina cedo fica bloqueado dentro de uma chamada coletiva, impedindo que a coleta do resultado comece antes que o último processo chegue. O

desbalanceamento deixa de ser apenas desperdício e passa a ser atraso imposto a todos.

Os números tornam esse ponto concreto. Com dezesseis processos no input padrão, a decomposição contígua atingiu speedup de 3,59 com fator de balanceamento de 0,9969, ao passo que o esquema master-worker alcançou 11,35 com fator de 0,0262. No caso de desbalanceamento acentuado a ordenação se manteve, com o master-worker atingindo 10,95. Para o mesmo problema, o mesmo hardware e o mesmo núcleo de cálculo, a escolha da estratégia de distribuição mais do que triplicou o ganho obtido.

A análise do custo de comunicação trouxe uma conclusão que inicialmente contrariava a expectativa. Esperava-se que a coleta da imagem fosse um gargalo relevante em memória distribuída, especialmente nas resoluções maiores. O custo líquido de comunicação, porém, permaneceu pequeno em todos os cenários do input padrão. O que parecia custo de comunicação, quando medido pelo tempo acumulado em chamadas de MPI, revelou-se tempo ocioso de espera causado pelo próprio desbalanceamento. Separar essas duas parcelas, conforme a decomposição apresentada em aula, foi o que permitiu diagnosticar corretamente o gargalo em vez de atribuí-lo à infraestrutura de troca de mensagens.

Um achado metodológico merece destaque por ter valor além desta etapa. O fator de balanceamento das estratégias estáticas não chegou a zero mesmo quando a quantidade de trabalho atribuída a cada processo era praticamente idêntica. Como a carga era equivalente, a diferença de tempo só podia vir da velocidade do núcleo em que cada processo foi alocado. Isso evidenciou que, em um processador híbrido, o balanceamento dinâmico opera em dois níveis ao mesmo tempo, diluindo a heterogeneidade do problema e compensando automaticamente a heterogeneidade do hardware. Nenhuma estratégia estática é capaz do segundo efeito, pois decide a atribuição antes de a execução começar.

A escalabilidade fraca, por sua vez, mostrou que o programa não é plenamente escalável neste hardware pelo critério de Gustafson adotado na disciplina, uma vez que nenhuma estratégia manteve a eficiência constante ao longo de toda a faixa. A dispersão entre as estratégias no mesmo ponto, contudo, indicou que a parcela dominante dessa perda tem origem na distribuição do trabalho, e que a limitação arquitetural atua como um piso adicional e não como causa única.

Do ponto de vista metodológico, três decisões se mostraram determinantes para a confiabilidade dos resultados. A primeira foi fixar o governador de frequência em performance, sem a qual os tempos ficaram cerca de 37% mais lentos e incomparáveis com os da Etapa 1. A segunda foi medir um baseline sequencial próprio para cada resolução da escalabilidade fraca, sem o qual não seria possível aplicar a fórmula de eficiência da disciplina nesse cenário. A terceira foi instrumentar o tempo de computação e o tempo em MPI por processo, o que permitiu ir além do tempo total e diagnosticar efetivamente onde o desbalanceamento acontece. Sem essa granularidade

de medição, teria sido fácil concluir apenas que uma estratégia é mais rápida que outra, sem compreender o porquê.

6. Referências

- UESC , Universidade Estadual de Santa Cruz. Notas de aula e enunciado da disciplina DEC107 , Processamento Paralelo, 2026.2.
- ETVORELLANA. RepoDEC107-PP-2026_2, Exemplos e exercícios da disciplina DEC107-PP. GitHub, 2026. Disponível em: https://github.com/etvorellana/RepoDEC107-PP-2026_2.
- MESSAGE PASSING INTERFACE FORUM. MPI: A Message-Passing Interface Standard. Disponível em: https://www.mpi-forum.org/docs/.
- THE OPEN MPI PROJECT. Open MPI Documentation. Disponível em: https://docs.open-mpi.org/.
- AMDAHL, G. Validity of the single processor approach to achieving large scale computing capabilities. Proceedings of the April 18-20, 1967, Spring Joint Computer Conference, p. 483-485, 1967.
- GUSTAFSON, J. L. Reevaluating Amdahl's law. Communications of the ACM, v. 31, n. 5, p. 532-533, 1988.
- PEITGEN, H.-O.; JÜRGENS, H.; SAUPE, D. Chaos and Fractals: New Frontiers of Science. 2. ed. New York: Springer, 2004. Referência de apoio aos conceitos teóricos do Conjunto de Mandelbrot e do método de escape-time apresentados na Introdução.
- RODRIGUES, C. S.; ORIHUELA, L. B. Relatório da Etapa 1, Benchmark de Geração do Conjunto de Mandelbrot, Implementação sequencial e paralela em memória compartilhada com OpenMP. DEC107, UESC, 2026.2. Fonte dos dados de OpenMP utilizados na comparação entre paradigmas apresentada na Seção 3.5.

DECLARAÇÃO DE TRANSPARÊNCIA E USO DE INTELIGÊNCIA ARTIFICIAL

Eu, Cibelle Sousa Rodrigues, e eu, Lucas Braga Orihuela, regularmente matriculados no curso de Ciência da Computação, declaramos para os devidos fins que, na elaboração do trabalho intitulado "Benchmark de Geração do Conjunto de Mandelbrot (Escape-Time): Etapa 2, MPI", submetido como requisito para a disciplina DEC107, Processamento Paralelo, adotamos as seguintes práticas em relação ao uso de ferramentas de Inteligência Artificial generativa:

(Assinale a opção aplicável)

( ) Não utilizei ferramentas de Inteligência Artificial na concepção, desenvolvimento, programação ou redação deste trabalho.

(X) Utilizei ferramentas de Inteligência Artificial (ex: ChatGPT, GitHub Copilot, Claude, etc.) como recursos de apoio.

Detalhamento do Uso: A ferramenta utilizada foi Claude, da Anthropic. O uso se deu exclusivamente nas seguintes etapas do projeto:

- Apoio na depuração (debugging) do código-fonte quando necessário.
- Geração do README do repositório e apoio na revisão/comentário do código-fonte.
- Revisão dos resultados apresentados na Seção 3 com relação aos experimentos feitos na máquina.
- Apoio no entendimento dos resultados obtidos na Seção 3.
- Apoio na produção de scripts para automatização dos experimentos.
- Revisão metodológica da coleta, que identificou a necessidade de medir um baseline sequencial próprio para cada resolução da escalabilidade fraca e a necessidade de fixar o governador de frequência em performance para manter a comparabilidade com a Etapa 1.

Termo de Responsabilidade: Declaramos estar cientes de que ferramentas de IA não são isentas de erros, vieses ou alucinações. Sendo assim, atestamos que toda a responsabilidade pela autoria, integridade, originalidade e exatidão do conteúdo submetido é exclusivamente nossa. Confirmamos que todo o código-fonte, dados de testes e textos gerados ou modificados com o auxílio de IA foram rigorosamente revisados, testados e validados por nós, garantindo que os resultados refletem o nosso próprio entendimento e esforço intelectual, sem incorrer em plágio intelectual ou acadêmico.

Ilhéus - BA, 30/10/2026

Cibelle Sousa Rodrigues
Matrícula: 202310768

Lucas Braga Orihuela
Matrícula: 202311125